

TÀI LIỆU CHUẨN BỊ TEAM

Vietnam AI Innovation Challenge 2026

Phiên bản: V1.0 – Giai đoạn chuẩn hóa tool, workflow và cách làm việc nhóm

1. Mục tiêu của tài liệu

Tài liệu này dùng để thống nhất cách team làm việc trước khi bước vào giai đoạn chọn đề tài, mock hackathon và thi chính thức.

Mục tiêu hiện tại chưa phải là làm sản phẩm cụ thể, mà là xây dựng một “hệ điều hành làm việc” cho team, bao gồm:

- Bộ công cụ sử dụng chung.
- Kiến trúc kỹ thuật dự kiến.
- Cách chia module.
- Cách dùng AI coding.
- Cách viết context cho AI.
- Cách quản lý code, merge code và tránh xung đột.
- Cách chuẩn bị demo, fallback và kiểm thử.
- Quy trình luyện tập trước ngày thi.

Tinh thần chung:

Team không chỉ cần biết code, mà cần biết phối hợp nhanh, rõ, ít lỗi và có khả năng biến ý tưởng thành demo chạy được trong thời gian ngắn.

2. Bản chất cuộc thi cần hiểu

Vietnam AI Innovation Challenge 2026 là một hackathon AI-native. Điều này có nghĩa là sản phẩm cần thể hiện AI nằm ở lõi giải pháp, không phải chỉ gắn chatbot hoặc AI vào cho có.

Team cần chuẩn bị theo hướng:

- Không làm sẵn một sản phẩm cố định 100% rồi đem đi demo.
- Nên chuẩn bị sẵn template, starter repo, workflow, component, module AI/RAG/API để khi có bài toán thật thì tùy biến nhanh.

- Mục tiêu trong hackathon là tạo ra một MVP có flow rõ, demo được, giải quyết một vấn đề cụ thể.
- Sản phẩm cần có tính thực tế, có người dùng mục tiêu, có pain point, có luồng xử lý và có giá trị rõ ràng.

Cách hiểu ngắn gọn:

Không chuẩn bị “sản phẩm đóng hộp”, mà chuẩn bị “xưởng sản xuất sản phẩm AI”.

3. Bộ công cụ team thống nhất sử dụng

3.1. Công cụ quản lý và giao tiếp

Team thống nhất dùng:

- GitHub: lưu trữ code, quản lý version, branch, pull request.
- Zalo: trao đổi nhanh, gửi thông báo, trao đổi ngắn.
- Google Docs hoặc Notion/OneNote: lưu tài liệu, kế hoạch, task, quyết định.
- Canva: làm slide pitch, hình minh họa, demo flow.

Ghi chú:

- Zalo chỉ dùng để chat nhanh. Những quyết định quan trọng cần được ghi lại vào Docs/Notion/OneNote.
 - Code bắt buộc lưu trên GitHub.
 - Tài liệu kỹ thuật, API contract, context pack nên nằm trong repo hoặc Docs chính thức của team.
 - Nếu dùng Notion/OneNote để giao task thì vẫn cần liên kết task với branch hoặc module code tương ứng.
-

3.2. Công cụ AI coding và AI hỗ trợ

Team hiện có và thống nhất sử dụng:

- ChatGPT Plus
- Codex
- Gemini / Antigravity
- Claude Sonnet Thinking

- Claude Opus Thinking

Cách dùng đề xuất:

Nhiệm vụ	Công cụ/model phù hợp
Phân tích đề, chia module, thiết kế kiến trúc	ChatGPT, Claude Opus
Viết code nhanh, sinh boilerplate	Codex, Gemini, Antigravity
Refactor module lớn	Claude Sonnet, Claude Opus
Debug lỗi khó	Claude Sonnet, ChatGPT, Codex
Viết tài liệu, context pack, prompt chuẩn	ChatGPT
Review code, review logic, kiểm tra kiến trúc	Claude Opus, ChatGPT
Viết pitch, mô tả sản phẩm, demo script	ChatGPT, Gemini

Quy tắc quan trọng:

AI coding không được tự ý đổi kiến trúc. AI chỉ được sửa trong phạm vi task, context pack và module contract đã được giao.

3.3. Công cụ codebase context / code graph

Team thống nhất dùng:

- GitNexus
- CodeGraph
- Repomix

GitNexus dùng để làm gì?

GitNexus dùng để nhìn repo theo dạng graph, hỗ trợ hiểu mối quan hệ giữa các file/module/function.

Dùng GitNexus khi:

- Thành viên mới cần hiểu repo.
- Cần xem module này liên quan đến module nào.
- Cần chuẩn bị trước khi merge code.
- Cần giải thích kiến trúc repo cho cả team.
- Cần tìm nhanh các file liên quan đến một flow.

Vai trò chính:

GitNexus = bản đồ trực quan để người trong team hiểu repo.

CodeGraph dùng để làm gì?

CodeGraph dùng để hỗ trợ AI coding hiểu codebase tốt hơn theo dạng graph/call relationship/symbol relationship.

Dùng CodeGraph khi:

- AI cần sửa code liên quan nhiều file.
- Cần xem function/class được gọi ở đâu.
- Cần refactor mà không phá flow cũ.
- Cần tìm impact của một thay đổi.
- Cần hỗ trợ AI không tự đoán code.

Vai trò chính:

CodeGraph = bộ nhớ cấu trúc cho AI coding agent.

Repomix dùng để làm gì?

Repomix dùng để đóng gói repo hoặc một phần repo thành file context để đưa vào ChatGPT/Claude/Gemini.

Dùng Repomix khi:

- Muốn đưa context của repo vào AI để hỏi.
- Muốn review kiến trúc.
- Muốn tạo snapshot trước khi refactor.
- Muốn chỉ đóng gói một module cụ thể thay vì toàn bộ repo.

Ví dụ:

```
npx repomix
```

```
npx repomix --style markdown
```

```
npx repomix --include "apps/api/**,packages/ai-core/**"
```

Vai trò chính:

Repomix = đóng gói context codebase để AI đọc đúng phần cần thiết.

4. Stack kỹ thuật đã chốt

Team chọn kiến trúc:

Next.js frontend + FastAPI backend + Supabase/PostgreSQL

Triển khai:

- Frontend: Next.js
 - UI: Tailwind CSS + shadcn/ui
 - Icon: Lucide Icons
 - Chart: Recharts
 - Backend: FastAPI
 - Database: Supabase PostgreSQL
 - Vector: pgvector trong Supabase trước, Qdrant chỉ dùng nếu cần nâng cấp
 - Storage: Supabase Storage hoặc lưu local tạm cho demo
 - Deploy frontend: Vercel
 - Deploy backend: Railway hoặc Render
 - Docs API: Swagger/OpenAPI từ FastAPI
-

5. Lý do chọn kiến trúc này

Team không chọn Next.js fullstack hoàn toàn vì sản phẩm AI-native có thể cần xử lý Python, RAG, OCR, CV hoặc model riêng.

FastAPI phù hợp hơn cho:

- AI service.
- RAG pipeline.
- CV/OCR module.
- Xử lý file.
- Kết nối model Python.
- Viết API rõ ràng.
- Tự sinh Swagger/OpenAPI.

Next.js phù hợp cho:

- UI.

- Dashboard.
- Chat interface.
- Upload screen.
- Report screen.
- Demo nhanh.
- Deploy Vercel.

Supabase phù hợp cho:

- PostgreSQL.
- Auth nếu cần.
- Storage nếu cần.
- pgvector cho RAG.
- Dễ setup nhanh trong hackathon.

Kiến trúc tổng quát:

User



Next.js Frontend



FastAPI Backend



AI/RAG/CV Services



Supabase PostgreSQL + pgvector + Storage

6. Cấu trúc repo đề xuất

Repo nên có cấu trúc như sau:

vnai-2026-starter/

|— apps/

| |— web/ # Next.js frontend

| |— api/ # FastAPI backend

```
|
|
|— packages/
|   |— ui/           # shared UI components nếu cần
|   |— prompts/      # prompt templates
|   |— shared-types/  # type/interface dùng chung
|   └— config/        # config chung nếu cần
|
|
|— docs/
|   |— architecture.md
|   |— api-contract.md
|   |— team-workflow.md
|   |— ai-coding-standard.md
|   |— demo-script.md
|   └— decisions.md
|
|
|— ai-context/
|   |— AGENTS.md
|   |— PROJECT_CONTEXT.md
|   |— CODING_RULES.md
|   |— MODULE_MAP.md
|   |— PROMPT_LIBRARY.md
|   └— context-packs/
|
|
|— data/
|   |— samples/
|   |— seed/
|   └— eval/
|
|
|— scripts/
```

```
| |— setup-dev/
| |— seed-db/
| |— run-eval/
| |— reset-demo/
|
|— .github/
| |— pull_request_template.md
| |— workflows/
|
|— README.md
|— docker-compose.yml
|— .env.example
```

7. Vai trò các thư mục quan trọng

apps/web

Chứa frontend Next.js.

Phụ trách:

- Layout.
- UI.
- Chat screen.
- Upload screen.
- Dashboard.
- Report screen.
- Gọi API backend.

apps/api

Chứa backend FastAPI.

Phụ trách:

- API endpoint.

- RAG service.
 - AI service.
 - File processing.
 - Database connection.
 - Vector search.
 - CV/OCR integration nếu có.
-

docs

Chứa tài liệu chính thức của team.

Các file quan trọng:

- architecture.md: kiến trúc tổng thể.
 - api-contract.md: thống nhất request/response giữa FE và BE.
 - team-workflow.md: quy trình làm việc nhóm.
 - ai-coding-standard.md: cách dùng AI coding.
 - decisions.md: ghi lại quyết định quan trọng.
-

ai-context

Đây là thư mục rất quan trọng.

Mục đích:

- Giúp AI coding hiểu project.
- Giúp team thống nhất cách viết code.
- Tránh AI tự đoán kiến trúc.
- Tránh mỗi người viết một kiểu.

Các file chính:

- AGENTS.md
- PROJECT_CONTEXT.md
- CODING_RULES.md
- MODULE_MAP.md
- PROMPT_LIBRARY.md

- context-packs/
-

8. Quy tắc chia module

Team chia việc theo module. Mỗi module cần có:

1. Owner chính.
2. Người hỗ trợ nếu cần.
3. Input.
4. Output.
5. API hoặc integration point.
6. File/folder phụ trách.
7. Tiêu chí hoàn thành.

Nguyên tắc:

- Module độc lập thì một người phụ trách chính.
- Module giao thoa thì cần có contract rõ.
- Không để nhiều người sửa cùng một file lớn nếu không cần thiết.
- Người phụ trách module có quyền quyết định nội bộ module, nhưng không được phá API contract chung.
- Các phân chuyển giao giữa module bắt buộc ghi rõ input/output.

Ví dụ:

Module: AI/CV Analysis

Owner: Thành viên AI

Input: image file hoặc image_url

Output:

```
{  
  "label": "...",  
  "confidence": 0.91,  
  "explanation": "...",  
  "metadata": {}  
}
```

Integration point:

POST /api/analyze-image

Ví dụ:

Module: RAG Chat

Owner: Thành viên AI + Backend

Input:

```
{  
  "session_id": "...",  
  "message": "...",  
  "collection_id": "..."  
}
```

Output:

```
{  
  "answer": "...",  
  "sources": [...],  
  "suggested_actions": [...]  
}
```

Integration point:

POST /api/chat

9. API contract

Frontend và backend không được làm theo cảm tính. Phải có API contract trước.

Ví dụ file docs/api-contract.md:

POST /api/chat

Request:

```
{  
  "session_id": "string",  
  "message": "string",
```

```
"collection_id": "string"
}
```

Response:

```
{
  "answer": "string",
  "sources": [
    {
      "title": "string",
      "content": "string",
      "score": 0.87
    }
  ],
  "actions": [
    {
      "type": "create_checklist",
      "payload": {}
    }
  ]
}
```

Quy tắc:

- FE có thể mock API trước theo contract.
- BE phải trả đúng contract.
- Nếu muốn đổi contract, phải báo team và cập nhật file docs trước.
- Không được đổi response làm vỡ frontend mà không thông báo.

10. Quy trình Git

Branch chính

main

dev

feature/...

fix/...

docs/...

experiment/...

Ý nghĩa:

- main: bản ổn định, dùng để demo/deploy.
- dev: nơi gộp tính năng trước khi lên main.
- feature: nhánh tính năng.
- fix: nhánh sửa lỗi.
- docs: nhánh sửa tài liệu.
- experiment: nhánh thử nghiệm, không merge trực tiếp nếu chưa kiểm tra.

Quy tắc branch

Không push trực tiếp vào main.

Tên branch nên theo mẫu:

feature/chat-rag

feature/upload-document

feature/dashboard-ui

feature/auth

feature/report-generation

fix/cors-error

fix/chat-response-format

docs/api-contract

experiment/ocr-test

Quy tắc commit

Commit message nên rõ nghĩa.

Ví dụ tốt:

feat: add chat API endpoint

fix: resolve CORS issue for frontend

docs: update API contract for report module

refactor: split RAG service from chat route

ui: add dashboard metric cards

Ví dụ không tốt:

update

fix bug

done

code mới

abc

Quy tắc Pull Request

Mỗi PR cần ghi:

What changed?

- ...

Related module

- frontend / backend / ai / docs

How to test?

- ...

Screenshots or API response

- ...

Risk

- low / medium / high

Nguyên tắc:

- PR nhỏ thì merge nhanh.
 - PR lớn phải review kỹ.
 - PR ảnh hưởng API contract phải báo team.
 - PR ảnh hưởng demo flow phải test lại end-to-end.
-

11. Quy trình dùng AI coding

Trước khi dùng AI sửa code, cần làm theo quy trình:

1. Đọc task.
 2. Xác định module liên quan.
 3. Đọc AGENTS.md, MODULE_MAP.md, context pack liên quan.
 4. Dùng GitNexus/CodeGraph/Repomix nếu cần để tìm file liên quan.
 5. Đưa đúng file/context vào AI.
 6. Yêu cầu AI sửa trong phạm vi nhỏ.
 7. Chạy test/lint/manual check.
 8. Commit rõ ràng.
 9. Mở PR hoặc báo team nếu cần.
-

Prompt chuẩn khi dùng AI coding

You are working inside our hackathon codebase.

Before editing:

1. Read AGENTS.md, PROJECT_CONTEXT.md, MODULE_MAP.md, and the related context pack.
2. Do not invent new architecture.
3. Reuse existing utilities, types, services, and API contracts.
4. Keep changes minimal and localized.
5. If a function already exists, extend it instead of duplicating it.
6. Do not change unrelated files.
7. After editing, list changed files and explain how to test.

Task:

[ghi task cụ thể]

Relevant files:

[ghi file liên quan]

Expected behavior:

[ghi kết quả mong muốn]

Constraints:

[ghi ràng buộc]

Ví dụ prompt tốt

Implement POST /api/chat according to docs/api-contract.md.

Use the existing RAGService in apps/api/services/rag_service.py.

Do not create another chat endpoint.

Do not change frontend code.

Return answer, sources, and actions according to the API contract.

Add basic error handling.

Relevant files:

- apps/api/routes/chat.py
- apps/api/services/rag_service.py
- docs/api-contract.md

Ví dụ prompt không tốt

Làm giúp tôi chức năng chat.

Lý do không tốt:

- Không có module.
 - Không có file.
 - Không có API contract.
 - AI dễ tự tạo kiến trúc mới.
 - AI dễ sửa lan man.
-

12. Context pack

Mỗi module quan trọng nên có một context pack.

Ví dụ:

ai-context/context-packs/chat-rag.md

ai-context/context-packs/upload-document.md

ai-context/context-packs/dashboard.md

ai-context/context-packs/cv-analysis.md

ai-context/context-packs/report-generation.md

Mẫu context pack:

Context Pack: [Tên module]

1. Purpose

Module này dùng để làm gì?

2. Owner

Ai phụ trách chính?

3. Related files

- ...

- ...

4. Data flow

Input → xử lý → output

5. API contract

Endpoint:

Request:

Response:

6. Dependencies

Module này phụ thuộc vào gì?

7. Do not

- Không tạo endpoint trùng.
- Không đổi response format khi chưa cập nhật contract.
- Không bypass service chính.
- Không hard-code dữ liệu demo vào logic chính.

8. Common tasks

- ...
- ...

9. Testing checklist

- ...

13. Các file Markdown bắt buộc nên có

AGENTS.md

Mục đích:

- Quy định cách AI coding phải làm việc trong repo.
- Cho AI biết giới hạn.
- Cho team biết chuẩn chung.

Nội dung nên có:

AGENTS.md

You are assisting a hackathon team building an AI-native product.

Rules:

1. Do not invent new architecture.
 2. Follow PROJECT_CONTEXT.md, CODING_RULES.md, and MODULE_MAP.md.
 3. Respect API contracts in docs/api-contract.md.
 4. Keep changes minimal and localized.
 5. Reuse existing services/components.
 6. Do not duplicate functions/components.
 7. Do not change unrelated modules.
 8. Always explain changed files and testing steps.
 9. If requirements are unclear, make the safest minimal assumption and document it.
 10. Prioritize demo stability over perfect architecture.
-

PROJECT_CONTEXT.md

Mục đích:

- Mô tả project đang làm.
- Cho AI và thành viên mới hiểu sản phẩm.

Nội dung nên có:

PROJECT_CONTEXT.md

This project is a starter codebase for Vietnam AI Innovation Challenge 2026.

The goal is to quickly build AI-native MVPs for real-world problems.

Core capabilities:

- AI chat/copilot
- RAG over uploaded documents
- Dashboard
- Report generation
- Optional CV/OCR module
- Supabase database
- FastAPI AI/backend service
- Next.js frontend

Architecture:

Next.js frontend → FastAPI backend → Supabase/PostgreSQL/pgvector → AI services

CODING_RULES.md

Mục đích:

- Chuẩn hóa code style và cách sửa code.

Nội dung nên có:

CODING_RULES.md

General rules:

1. Keep code simple and readable.
2. Prefer working demo over over-engineered architecture.
3. Do not create duplicate utilities.
4. Do not hard-code secrets.
5. Use environment variables.
6. Keep API response format stable.
7. Update docs when changing contracts.
8. Add clear error handling for demo-critical flows.
9. Do not commit broken code to main.
10. Test manually before opening PR.

MODULE_MAP.md

Mục đích:

- Cho biết module nào nằm ở đâu, ai phụ trách, phụ thuộc vào gì.

Mẫu:

MODULE_MAP.md

Frontend

Path: apps/web

Owner: [Tên]

Responsibilities:

- UI
- Chat screen
- Upload screen
- Dashboard
- Report screen

Backend API

Path: apps/api

Owner: [Tên]

Responsibilities:

- FastAPI routes
- API contract
- Database connection
- Service orchestration

AI/RAG

Path: apps/api/services/rag

Owner: [Tên]

Responsibilities:

- Embedding
- Retrieval
- Prompting
- Answer generation
- Source citation

CV/OCR

Path: apps/api/services/cv

Owner: [Tên]

Responsibilities:

- Image processing
- OCR
- CV model
- Analysis result

Product/Demo

Path: docs, data/samples

Owner: [Tên]

Responsibilities:

- User flow
- Demo script
- Sample data
- Pitch content

DECISIONS.md

Mục đích:

- Ghi lại quyết định quan trọng.
- Tránh hỏi lại hoặc đổi lung tung.

- Giúp AI hiểu lý do chọn kiến trúc.

Mẫu:

DECISIONS.md

Decision 001: Use Next.js + FastAPI + Supabase

Date: ...

Decision:

Use Next.js for frontend, FastAPI for backend/AI service, and Supabase PostgreSQL for database.

Reason:

- Next.js is fast for frontend and deploys easily on Vercel.
- FastAPI is suitable for AI/RAG/CV modules in Python.
- Supabase provides PostgreSQL, storage, and pgvector.

Impact:

- Need to manage CORS.
- Need API contract between frontend and backend.
- Deploy frontend and backend separately.

Status:

Accepted

14. UI template cần chuẩn bị

Stack UI:

Next.js + Tailwind CSS + shadcn/ui + Lucide Icons + Recharts

Các màn hình nên có sẵn:

1. Landing / Problem intro

2. Chat Copilot
3. Upload Data / Knowledge Base
4. Dashboard
5. Analysis Result / Report
6. Settings / Data Sources
7. Action Checklist nếu cần
8. History nếu kịp

Component nên chuẩn bị:

- AppSidebar
- TopNav
- ChatWindow
- MessageBubble
- SourceCitationCard
- FileUploadBox
- DatasetTable
- MetricCard
- ChartCard
- ActionChecklist
- ReportPreview
- LoadingSkeleton
- EmptyState
- ErrorState

Nguyên tắc UI:

- Đẹp vừa đủ, sạch, dễ hiểu.
- Không quá màu mè.
- Demo flow phải rõ.
- Màn hình phải có dữ liệu mẫu.
- Có loading state và error state.
- Với RAG, cần hiển thị nguồn/citation.

- Với AI action, cần hiển thị checklist hoặc next action.
-

15. Evaluation và demo safety

Phần này tùy bài toán, nhưng team cần chuẩn bị tư duy từ đầu.

15.1. Demo data cố định

Không demo bằng dữ liệu ngẫu nhiên.

Cần có:

data/samples/

├── documents/

├── images/

├── csv/

├── sample_questions.md

└── sample_outputs.md

Mục tiêu:

- Demo ổn định.
 - Không phụ thuộc vào input lạ.
 - Dễ test lại.
 - Dễ quay video backup.
-

15.2. Sample questions

Với mỗi bài toán, chuẩn bị ít nhất:

1. Câu hỏi dễ, có câu trả lời rõ trong dữ liệu.
 2. Câu hỏi cần tổng hợp nhiều nguồn.
 3. Câu hỏi yêu cầu AI tạo checklist/action.
 4. Câu hỏi không có trong dữ liệu để kiểm tra hallucination.
 5. Câu hỏi dùng cho demo chính.
-

15.3. Fallback

Cần có phương án nếu:

- API LLM lỗi.
- Backend deploy bị sleep.
- Internet yếu.
- Vector search lỗi.
- Demo live không chạy.

Fallback cần chuẩn bị:

- Cached response cho demo path.
- Screenshot màn hình chính.
- Video demo 1–2 phút.
- Local mode.
- Seed data/reset script.

Nguyên tắc:

Demo live là chính, nhưng phải có backup để không “chết đứng” nếu lỗi kỹ thuật.

15.4. Demo script

Trước khi pitch cần có script:

Step 1: Giới thiệu pain point.

Step 2: Mở app.

Step 3: Upload hoặc chọn dữ liệu mẫu.

Step 4: Hỏi AI một câu quan trọng.

Step 5: Hiện thị câu trả lời + nguồn.

Step 6: Cho AI tạo checklist/action/report.

Step 7: Mở dashboard hoặc report.

Step 8: Kết luận impact.

Không demo tùy hứng.

16. Quy trình chuẩn bị theo giai đoạn

Giai đoạn 0: Chốt tool và workflow

Mục tiêu:

- Team hiểu dùng tool nào.
- Team hiểu repo sẽ vận hành ra sao.
- Team hiểu AI coding phải theo context.
- Team hiểu branch/PR/API contract.

Output:

- Tài liệu tool stack
 - Tài liệu workflow
 - AGENTS.md bản đầu
 - MODULE_MAP.md bản đầu
 - CODING_RULES.md bản đầu
-

Giai đoạn 1: Tạo repo starter

Mục tiêu:

- Có repo chạy được.
- Có frontend cơ bản.
- Có backend cơ bản.
- Có Supabase connected.
- Có API health check.
- Có README setup.

Output:

- Next.js app chạy được
 - FastAPI chạy được
 - Supabase connected
 - /health endpoint
 - .env.example
 - README setup
-

Giai đoạn 2: Mini task cá nhân

Mục tiêu:

- Mỗi người hiểu nhiệm vụ.
- Kiểm tra năng lực thực tế.
- Kiểm tra cách dùng AI coding.
- Kiểm tra cách commit/merge.

Ví dụ task:

FE member:

- Làm dashboard mock data.
- Làm chat UI.

AI member:

- Làm RAG basic trên 2–3 file mẫu.
- Trả về answer + sources.

Backend member:

- Làm upload API.
- Làm API contract cho chat.

Flow/Product member:

- Viết user journey.
- Viết demo script.
- Chuẩn bị sample data.

Giai đoạn 3: Đề tài giả AI-native nhỏ

Mục tiêu:

- Gộp module thật.
- Team làm quen end-to-end.
- Biết FE/BE/AI nối với nhau ra sao.

Đề tài giả đề xuất:

AI Copilot cho trung tâm tiếng Anh nhỏ:

- Upload tài liệu khóa học.
- Hỏi đáp thông tin khóa học.
- Phân loại học viên tiềm năng.
- Gợi ý checklist tư vấn.
- Tạo báo cáo ngắn cho nhân viên tư vấn.

Lý do chọn đề này:

- Có RAG.
- Có dashboard.
- Có dữ liệu mẫu.
- Có action/checklist.
- Có flow doanh nghiệp nhỏ.
- Không quá khó.
- Đúng tinh thần AI-native.

Giai đoạn 4: Mock hackathon áp lực

Mục tiêu:

- Giả lập gần giống cuộc thi.
- Phát hiện điểm yếu teamwork.
- Phát hiện lỗi merge.
- Phát hiện thiếu context pack.
- Phát hiện người nào hợp task nào.

Yêu cầu:

Thời gian: 6–12 tiếng

Output bắt buộc:

- Problem statement
- User flow
- UI demo
- Backend API
- AI feature

- Sample data

- Slide ngắn

- Demo script

Sau mock cần review:

- Ai bị nghẽn?

- Module nào gộp khó?

- AI coding lỗi ở đâu?

- API contract có thiếu không?

- Context pack có đủ không?

- UI có mất thời gian không?

- Demo có ổn định không?

Giai đoạn 5: Chuẩn bị template đa lĩnh vực

Sau khi team vận hành ổn, mới mở rộng template cho nhiều mảng.

Các template nên chuẩn bị:

1. AI Knowledge Copilot

Dùng cho giáo dục, dịch vụ công, nội bộ doanh nghiệp.

2. AI Business Operation Copilot

Dùng cho SME, CSKH, bán hàng, vận hành.

3. AI Document/OCR/CV Copilot

Dùng cho tài liệu scan, hình ảnh, hóa đơn, hồ sơ.

4. AI Report/Decision Assistant

Dùng cho dashboard, phân tích dữ liệu, tạo báo cáo.

17. Freeze rule trong hackathon

Trong hackathon thật, cần có mốc khóa tính năng.

Ví dụ nếu còn 6 tiếng trước khi nộp:

T-6h: Feature freeze

Không thêm tính năng lớn nữa.

T-4h: Demo freeze

Chốt flow demo, chỉ sửa lỗi liên quan demo.

T-2h: Slide freeze

Chốt slide, tập pitch, chuẩn bị backup.

T-1h: Stability only

Chỉ sửa bug nhỏ, không refactor, không đổi kiến trúc.

Nguyên tắc:

Gần giờ nộp thì ổn định quan trọng hơn thêm tính năng.

18. Quy tắc ưu tiên khi làm sản phẩm

Trong 48 giờ, ưu tiên theo thứ tự:

1. Demo chạy được.
2. Flow dễ hiểu.
3. AI có vai trò rõ.
4. UI đủ đẹp và sạch.
5. Có dữ liệu mẫu tốt.
6. Có pitch tốt.
7. Có khả năng mở rộng.
8. Tối ưu kỹ thuật sâu nếu còn thời gian.

Không ưu tiên:

- Kiến trúc quá phức tạp.
- Train model nặng nếu không cần.
- Quá nhiều tính năng.
- UI cầu kỳ nhưng không có backend.
- AI phức tạp nhưng demo không chạy.
- Refactor sát giờ nộp.

19. Định nghĩa MVP tốt cho team

Một MVP tốt trong cuộc thi nên có:

- Một người dùng cụ thể.
- Một vấn đề cụ thể.
- Một luồng demo chính.
- Một chức năng AI thật sự hữu ích.
- Một dashboard/report/result screen.
- Một dữ liệu mẫu đủ thuyết phục.
- Một số liệu impact đơn giản.
- Một hướng mở rộng sau cuộc thi.

Ví dụ impact:

- Giảm thời gian tra cứu tài liệu từ 10 phút xuống còn 30 giây.
- Tự động tạo checklist thay vì nhân viên phải soạn thủ công.
- Hỗ trợ phân loại khách hàng tiềm năng.
- Tóm tắt dữ liệu vận hành trong một màn hình.
- Trả lời có nguồn để giảm hallucination.

20. Kết luận

Bộ chốt hiện tại của team:

GitHub

Zalo

Canva

Docs / Notion / OneNote

ChatGPT Plus

Codex

Gemini / Antigravity

Claude

GitNexus

CodeGraph

Repomix

Next.js

Supabase

FastAPI

Vercel

Railway / Render

Kiến trúc chót:

Next.js frontend



FastAPI backend



Supabase PostgreSQL + pgvector



AI/RAG/CV services

Việc cần làm tiếp theo:

1. Tạo repo starter.
2. Tạo các file Markdown chuẩn:
 - AGENTS.md
 - PROJECT_CONTEXT.md
 - CODING_RULES.md
 - MODULE_MAP.md
 - DECISIONS.md
3. Chốt role từng người.
4. Chốt API contract mẫu.
5. Làm mini task cá nhân.
6. Chạy đề tài giả nhỏ.
7. Review điểm yếu.
8. Chạy mock hackathon áp lực.

9. Sau đó mới mở rộng template theo nhiều mảng.

Tinh thần cuối cùng:

Team không cần tool quá xịn. Team cần quy trình rõ, context tốt, module contract chặt, demo ổn định và khả năng tận dụng AI coding có kiểm soát.